

通用:

- 1.请描述一下你的项目?
- 2.你觉得在项目中遇到的最大的问题(难点)是什么?怎么解决的?
- 3.开发板的芯片是什么?搭载了 Linux 操作系统吗?主频多少?是 ARM 什么系列产品?
三星 s5p6818,搭载了 Linux3.2 版本,主频 1.4GHZ,Cortex-A53

图片相关:

- 3.怎么区分 bmp 和其他类型的图片?
- 4.BMP 图片是怎么显示的?
- 5.如何获取图片名称的,图片名是如何存储的?数组和链表的区别?
- 6.为什么要使用 mmap 映射屏幕?

触摸屏相关:

- 7.触摸屏点是怎么获取的?
- 8.如何获取/区分滑动方向的?

线程相关:

- 9.项目中用到了线程没有?解决了什么问题?
- 10.进程和线程有什么区别?
- 11.什么时候需要使用到进程间通信?信号量主要用来做什么?

网络相关:

- 12.项目里面使用的是 TCP 还是 UDP?有什么区别?
- 13.TCP 为什么是可靠的通信?
- 14.描述一下 3 次握手和 4 次挥手的过程?

电子音乐相册:

- 14.播放音乐的逻辑是怎么实现的?
- 15.可以同时播放音乐和显示相册吗?如何处理的
- 16.如果在播放相册的时候,想切换歌曲,怎么处理的?

五子棋:

- 17.游戏逻辑是怎么实现的?
- 18.如何判断游戏是否结束?能否联机对战?如果需要加入这个功能呢?
- 19.点击屏幕,如何确定落子的位置

FTP 文件传输:

- 20.里面用的传输层协议是什么?
- 21.可以传输目录吗?实现了什么功能?
- 22.传输大文件可以成功吗?遇到了什么问题?
- 23.有没有使用私有协议?私有协议大概怎么设计的?

必须要有私有协议!!!!!!没有私有协议统统不行

线程池目录拷贝:

- 24.线程池是什么?为什么要在拷贝目录中使用线程池,还可以用到什么地方?
- 25.任务队列用什么实现的?里面互斥锁用来保护什么?可以不使用吗?
- 25.如果需要增加一个线程或者删除一个线程,你怎么处理的?
- 26.如果打算取消一个线程,应该注意什么问题?

重点:

专业技能里面描述的技术点都需要搞懂!!!!

1、解释堆和栈的区别

栈区（**stack**）：由编译器自动分配释放，存放函数的参数值，局部变量的值等。

堆：一般由程序员分配释放，若程序员不释放，程序结束时可能由 OS 回收。

2、请比较标准 IO 和系统 IO 的区别

1) 标准 IO 是标准 C 库提供的一套对文件操作的接口，带缓冲区，只要开发环境中标准 C 库，即可使用标准 IO，可移植性强。标准 IO 只能操作普通文件

2) 系统 IO 是由操作系统内核提供的一套操作文件的接口，不带缓冲区，各个操作系统平台提供的系统 IO 接口可能不一样，可移植性差。系统 IO 能够操作普通文件和设备文件、管道文件、**socket** 文件

3、什么是可重入函数？哪些情况下的函数是不可重入的

一个可重入的函数简单来说就是可以被中断而不会造成影响的函数。

满足下列条件的函数多数是不可重入的：

- 1) 函数体内使用了静态的数据结构；
- 2) 函数体内调用了 **malloc()** 或者 **free()** 函数；
- 3) 函数体内调用了标准 I/O 函数。

4、什么是孤儿进程,什么是僵尸进程?

孤儿进程：一个父进程退出，而它的一个或多个子进程还在运行，那么那些子进程将成为孤儿进程。孤儿进程将被 **init** 进程(进程号为 1)所收养，并由 **init** 进程对它们完成状态收集工作。

僵尸进程：一个进程使用 **fork** 创建子进程，如果子进程退出，而父进程并没有调用 **wait** 或 **waitpid** 获取子进程的状态信息，那么子进程的进程描述符仍然保存在系统中。这种进程称之为僵尸进程。

僵尸进程的避免

- 1.父进程通过父进程会过 **wait** 和 **waitpid** 等函数等待子进程结束，这会导致父进程挂起。
2. 如果父进程很忙，那么可以用 **signal** 函数为 **SIGCHLD** 安装 **handler**，因为子进程结束后，收到该信号，可以在 **handler** 中调用 **waitpid** 回收。
3. 如果父进程不关心子进程什么时候结束，那么可以用 **signal (SIGCHLD,SIG_IGN)** 通知内核，自己对子进程的结束不感兴趣，那么子进程结束后，内核会回收，并不再给父进程发送信号。

5、请简述 linux 进程的地址空间布局

Linux 对进程的数据进行分段管理，不同的属性的数据，存储在不同的“内存段”中，不同的“内存段”的属性是不同的，so 不同的“内存段”的管理的方式也不一样。

.text

主要是用来存放代码。

只读并且共享。这段内存存在进程运行期间,不会释放的“代码段” 随进程持续性。

.data

数据段。

主要存放程序的 已经初始化的全局变量 和 已经初始化的 `static` 变量
可读可写。这段内存存在进程运行期间，一直存在，随进程持续性的。

`.bss`

数据段。

主要存放程序的 没有初始化的全局变量 和 没有初始化的 `static` 变量
可读可写。这段内存存在进程运行期间，一直存在，随进程持续性的。

`.bss`，在进程初始化的时候，(可能)全部初始化为 0。

`.rodata` :

只读数据段

主要存放程序中的只读数据的(如: 字符串常量)

只读。这段内存存在进程运行期间，一直存在。

随进程持续性。

栈(Stack)空间:

主要存放局部变量(非 `static` 的局部变量)

可读可写。这个段空间，会自动释放(代码块执行完啦，代码中的局部
变量的空间就会被自动释放)。随代码块持续性的。

堆(Heap)空间: 动态内存空间

主要是 `malloc/realloc/calloc` 动态分配的空间。

可读可写。这段内存存在进程运行期间，一旦分配，

就会一直存在，直到你手动 `free` 或进程消亡。

防止 "内存泄漏" / "垃圾内存"

6、请简述 gcc 的编译过程？

gcc 编译过程分为 4 个阶段：预处理、编译、汇编、链接。

预处理：处理以#开头的行, 主要有头文件包含、宏替换、条件编译、删除注释

编译：主要进行词法、语法、语义分析等，检查无误后将预处理好的文件编译成汇编文件。

汇编：将汇编文件转换成 二进制目标文件

7、中断是嵌入式系统中重要的组成部分，这导致了很多编译开发商提供一种扩展—让标准 C 支持中断。具代表事实是，产生了一个新的关键字 `__interrupt`。下面的代码就使用了 `__interrupt` 关键字去定义了一个中断服务子程序 (ISR)，请评论一下这段代码的。

```
__interrupt double compute_area (double radius)
{
    double area = PI * radius * radius;
    printf("\nArea = %f", area);
    return area;
}
```

这个函数有太多的错误了：

1. ISR 不能返回一个值。如果你不懂这个，那么你不会雇佣的。
2. ISR 不能传递参数。如果你没有看到这一点，你被雇佣的机会等同第一项。
3. 在许多的处理器/编译器中，浮点一般都是不可重入的。有些处理器/编译器需要让额处的寄存器入栈，有些处理器/编译器就是不允许在 ISR 中做浮点运算。此外，ISR 应该是

短而有效率的，在 ISR 中做浮点运算是明智的。

4. 与第三点一脉相承，printf() 经常有重入和性能上的问题。如果你丢掉了第三和第四点，不会太为难你的。不用说，如果你能得到后两点，那么你的被雇用前景越来越光明了。

8、请列举 ISO/OSI 层参考模型和 TCP/IP 协议 4 层模型有哪些，并简述其对应关系

OSI 模型共有七层，从下到上分别是物理层、数据链路层、网络层、传输层、会话层、表示层和应用层。

TCP/IP 协议包括四个层次：网络接口层、网络层、传输层、应用层。



应用层：应用层是 TCP/IP 协议的第一层，是直接为应用进程提供服务的。

(1) 对不同种类的应用程序它们会根据自己的需要来使用应用层的不同协议，邮件传输应用使用了 SMTP 协议、万维网应用使用了 HTTP 协议、远程登录服务应用使用了有 TELNET 协议。

(2) 应用层还能加密、解密、格式化数据。

(3) 应用层可以建立或解除与其他节点的联系，这样可以充分节省网络资源。

运输层：作为 TCP/IP 协议的第二层，运输层在整个 TCP/IP 协议中起到了中流砥柱的作用。且在运输层中，TCP 和 UDP 也同样起到了中流砥柱的作用。

网络层：网络层在 TCP/IP 协议中的位于第三层。在 TCP/IP 协议中网络层可以进行网络连接的建立和终止以及 IP 地址的寻找等功能。

网络接口层：在 TCP/IP 协议中，网络接口层位于第四层。由于网络接口层兼并了物理层和数据链路层所以，网络接口层既是传输数据的物理媒介，也可以为网络层提供一条准确无误的线路。

七层模型各层的作用：

应用层：应用层是 OSI 参考模型的最高层，是用户与网络的接口。

表示层：表示层处理流经结点的数据编码的表示方式问题，以保证一个系统应用层发出的信息可被另一系统的应用层读出。数据格式化、数据加密。

会话层：会话层主要功能是管理和协调不同主机上各种进程之间的通信（对话），即负责建立、管理和终止应用程序之间的会话建立、管理和维护会话。

传输层：传输层的作用是为上层协议提供端到端的可靠和透明的数据传输服务，包括处理差错控制和流量控制等问题。

网络层：网络层是为传输层提供服务的，传送的协议数据单元称为数据包或分组。

数据链路层：数据链路层是为网络层提供服务的，解决两个相邻结点之间的通信问题。

物理层：物理层是 OSI 参考模型的最低层，它利用传输介质为数据链路层提供物理连接。

众所周知，OSI 参考模型是学术上和法律上的国际标准，是完整的权威的网络参考模型。而 TCP/IP 参考模型是事实上的国际标准，即现实生活中被广泛使用的网络参考模型。

9、请简述进程间通信有哪些方式，各有什么特点(10 分)

1，管道，分为有名管道和无名管道

共同特点：

1. 一端读，一端写
2. 按顺序读(不支持 lseek)
3. 数据读走了，就没有了
4. 只存在于内存，随内核持续性

区别：

有名管道可用于所有进程，无名管道只可用于有亲缘关系的进程。

2，消息队列

特点：

- 1 只存在于内存，随内核持续性
- 2, 可以通过顺序和消息类型读取，也可以 fifo 读取
- 3, 消息队列可实现双向通信。

3，信号量

特点：

- 1, 在内核中创建一个信号量集合（本质是个数组）
- 2, 通过设置代码临界区来保护共享资源

4，共享内存

特点：

- 1，将内核空间映射到不同的进程的空间中，实现不同进程间对同一资源的共享。
- 2，不用从用户态到内核态的频繁切换和拷贝数据，直接从内存中读取就可以，
- 3，共享内存是共享资源，需要保护

5，信号

特点：

- 1，软中断机制，提供了一种处理异步事件的方法
- 2，只能进行整数的数据通信

6，Socket

特点：

利用 Socket 接口来实现的一种高效的进程间通信方式。

10、简述一下进程的三态模型

按进程在执行过程中的不同情况至少要定义三种不同的进程状态：

- (1) 运行(running)态：进程占有处理器正在运行。
- (2) 就绪(ready)态：进程具备运行条件，等待系统分配处理器以便运行。
- (3) 等待(wait)态：又称为阻塞(blocked)态或睡眠(sleep)态，指进程不具备运行条件

11、请简述进程与线程之间有什么区别？(10 分)

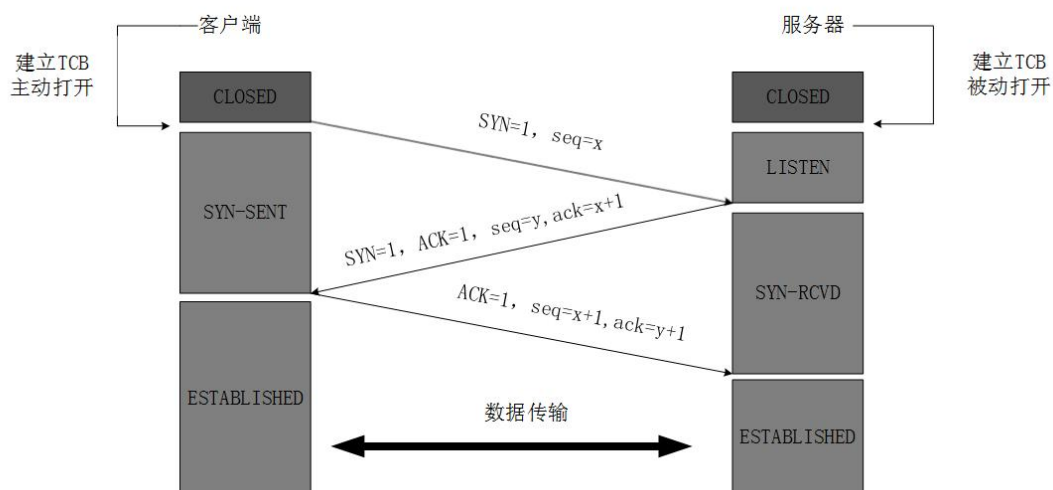
- 1) 进程是操作系统资源分配的最小单位，线程是调度的最小单位。
- 2) 创建一个线程比创建一个进程开销要小的多。
- 3) 实现线程间通信十分方便，通信开销小。

4) 线程是进程中的一个执行路径，且每个进程都会有一个主线程。

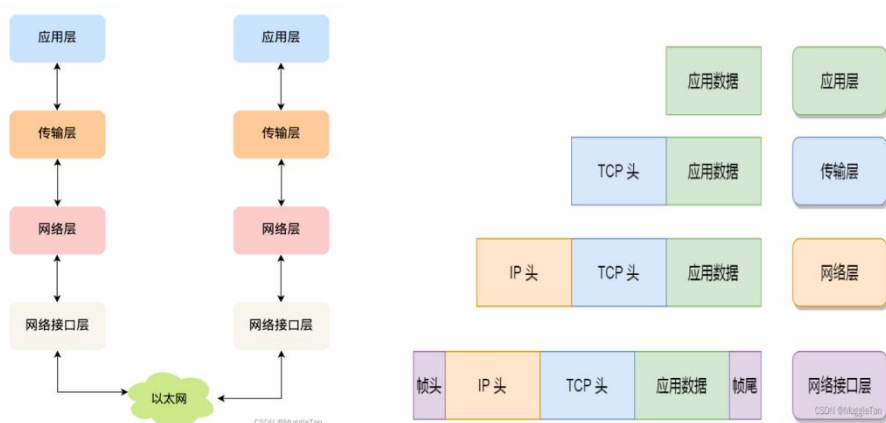
12、请简述造成死锁的四个必要条件

- 1) 互斥条件
- 2) 请求与保持条件
- 3) 不可剥夺条件
- 4) 循环等待条件

13、请画出三次握手过程图



14、请图出 TCP/IP 四层协议模型和每一层的封装结构



常用网络协议神图

⑦应用层

POP3	邮局协议版本3	110
SMTP	简单邮件传输协议	25
Telnet	远程控制	23
FTP	文件传输协议	21
HTTP	超文本传输协议	80
IMAP	因特网消息访问	143
HTTP-s	HTTP安全协议	443

DHCP	动态主机配置协议	67
TFTP	简单文件传输协议	69
SNMP	简单网络管理协议	161
SLP	服务定向协议	
NTP	网络时间协议	
RADIUS	远程用户拨号认证	
BOOTP	引导程序协议	

DNS

53

⑥表示层

⑤会话层

LDAP
轻量级目录访问协议

7号信令系统

SS7

④传输层

DVMRP
距离矢量多播选路协议

MOBILE IP 移动IP

SSL/TLS 安全套接字协议

TCP

传送适配层协议

TALI

UDP

③网络层

BGP 边界网关协议 179

HSRP 热备份路由协议 1985
RIP/RIPng 距离矢量路由协议 520/521

IGMP	因特网组管理协议	2
ICMP	因特网控制报文协议	1
RSVP	资源预留协议	46
ESP	IP安全封装协议	50
AH	IP安全认证协议	51
X.25	分组交换网协议	93

OSPF	开放最短路径优先	89
IS-IS	中间系统到中间系统协议	124
EGP	外部网关路由协议	8
VRRP	虚拟路由冗余协议	112
IGRP	动态距离矢量路由协议	9
EIGRP	增强IGRP路由协议	88

路由协议

IP

6

17

②链路层

802.1d	冗余链路STP
802.1w	快速STP
802.1q	VLAN
802.1x	认证系统
802.1p	Qos流量优先级
802.1g	远程网桥

CSMA/CA

IEEE 802.1

IEEE 802.11

IEEE 802

IEEE 802.3

CSMA/CD

HDLC 高级链路控制
SDLC 同步链路控制

PPP 点对点协议
PPPOE 以太网PPP

L2F 第二层转发协议
L2TP 第二层隧道协议
PPTP 点对点隧道协议

VPN协议

LACP 链路汇聚控制协议 802.3ad
ISL 思科交换机链路协议 已淘汰

ARP 地址解析协议
RARP 反向地址解析协议

ATM
异步传输网

①物理层

专线	E1/E3	接入网	T1/T3
窄带	PSTN	ISDN	X.25
宽带	Ether	ADSL	EPON
传输	SDH	RPR	MSTP
无线	GPRS	3G	4G

WLAN标准

802.11b	2.4GHZ	11Mbps
802.11a	5GHZ	54Mbps
802.11g	2.4GHZ	54Mbps
802.11n	2.4/5G	300~600
802.11ac	5GHZ	500~1G
IEEE802.15	(蓝牙技术)	
IEEE802.16	(无线城域网)	

MLAN标准

IEEE802.3ab	1000M
1000Base-T	四对UTP
IEEE802.3ae	10G
10GBase-S	多模光纤 300m
10GBase-L	单模光纤 10Km
10GBase-E	单模光纤 40Km
10GBase-LX4	单模光纤 10Km
IEEE802.3an	10G
100Base-T	双绞线 100m
IEEE802.3ba	100G 光纤

LAN标准

IEEE 802.3z	1000M
1000Base-SX	短波光纤 550m
1000Base-LX	长波光纤 5000m
1000Base-CX	两对STP 25m
IEEE 802.3u	100M
100Base-TX	两对五类UTP 100m
100Base-FX	一对单模光纤SMF 40km
100Base-T4	四对3类UTP (淘汰) 100m

1、项目描述

请先整体描述项目实现的功能
再按模块分别描述

2、简述一下C++中的多态

在现实生活中，多态是同一个事物在不同场景下的多种形态。

在面向对象中，多态是指通过基类的指针或者引用，在运行时动态调用实际绑定对象函数的行为，与之相对应的编译时绑定函数称为静态绑定。所以多态分为静态多态和动态多态。

1. 静态多态 静态多态是编译器在编译期间完成的，编译器会根据实参类型来选择调用合适的函数，如果有合适的函数就调用，没有的话就会发出警告或者报错。静态多态有函数重载、运算符重载、泛型编程等。
2. 动态多态 动态多态是在程序运行时根据基类的引用（指针）指向的对象来确定自己具体该调用哪一个类的虚函数。当父类指针（引用）指向 父类对象时，就调用父类中定义的虚函数；即当父类指针（引用）指向 子类对象时，就调用子类中定义的虚函数。

3、简述一下深拷贝与浅拷贝

1. 浅拷贝 浅拷贝又称为值拷贝，将源对象的值拷贝到目标对象中，如果对象中有某个成员是指针类型数据，并且是在堆区创建，则使用浅拷贝仅仅拷贝的是这个指针变量的值，也就是在目标对象中该指针类型数据和源对象中的该成员指向的是同一块堆空间。这样会带来一个问题，就是在析构函数中释放该堆区数据，会被释放多次。默认的拷贝构造函数和默认的赋值运算符重载函数都是浅拷贝。
2. 深拷贝 深拷贝在拷贝的时候先开辟出和源对象大小一样的空间，然后将源对象里的内容拷贝到目标对象中去，这样指针成员就指向了不同的内存位置。并且里面的内容是一样的，这样不但达到了拷贝的目的，还不会出现问题，两个对象先后去调用析构函数，分别释放自己指针成员所指向的内存。深拷贝情况下，不会出现重复释放同一块内存的错误。

4、STL容器用过哪些？查找的时间复杂度是多少？

STL 中常用的容器有 vector、deque、list、map、set、multimap、multiset、unordered_map、unordered_set 等。

容器底层实现方式及时间复杂度分别如下：

1. vector 采用一维数组实现，元素在内存连续存放，不同操作的时间复杂度为：插入: $O(N)$ 查看: $O(1)$ 删除: $O(N)$
2. deque 采用双向队列实现，元素在内存连续存放，不同操作的时间复杂度为：插入: $O(N)$ 查看: $O(1)$ 删除: $O(N)$
3. list 采用双向链表实现，元素存放在堆中，不同操作的时间复杂度为：插入: $O(1)$ 查看: $O(N)$ 删除: $O(1)$
4. map、set、multimap、multiset 上述四种容器采用红黑树实现，红黑树是平衡二叉树的一种。不同操作的时间复杂度近似为：插入: $O(\log N)$ 查看: $O(\log N)$ 删除: $O(\log N)$
5. unordered_map、unordered_set、unordered_multimap、unordered_multiset 上述四种容器采用哈希表实现，不同操作的时间复杂度为：插入: $O(1)$ ，最坏情况 $O(N)$ 查看: $O(1)$ ，最坏情况 $O(N)$ 删除: $O(1)$ ，最坏情况 $O(N)$
注意：容器的时间复杂度取决于其底层实现方式。

5、说说STL中容器的类型，每种分别有哪些容器？

1. 序列式容器 array、vector、deque、list、forward_list
2. 关联式容器 map、multimap、set、multiset
3. 无序关联式容器 unordered_map、unordered_multimap、unordered_set、unordered_multiset
4. 容器适配器 stack、queue、priority_queue

6、简述一下C++的重载与重写

- 重载

- a. 重载是指不同的函数使用相同的函数名，但是函数的参数个数或类型不同（参数列表不同）。调用的时候根据函数的参数来区别不同的函数，函数重载跟返回值无关。
- b. 重载的规则 - 作用域相同 - 函数名相同 - 必须具有不同的参数列表
- c. 重载用来实现静态多态（函数名相同，功能不一样）。

- 重写

- a. 重写（也叫覆盖）是指在派生类中重新对基类中的虚函数重新实现。即函数名和参数都一样，只是函数的实现体不一样。
- b. 重写的规则： - 方法声明必须完全与父类中被重写的方法相同 - 子类重写的方法可以加virtual，也可以不加
- c. 重写用来实现动态多态（根据调用方法的对象的类型来执行不同的函数）。

7、为什么将析构函数设置成虚函数？

1. 概念--虚析构函数，是将基类的析构函数声明为 virtual ,如：

```
class Base
{
    public: Base() { }
    // 虚析构函数
    virtual ~Base() { }
}
```

2. 作用--虚析构函数的主要作用是为了防止遗漏资源的释放，防止内存泄露。

如果基类中的析构函数没有声明为虚函数，基类指针指向派生类对象时，则当基类指针释放时不会调用派生类对象的析构函数，而是调用基类的析构函数，如果派生类析构函数中做了某些释放资源的操作，则这时就会造成内存泄露。

8、说说map和set的区别

map和set都是C++的关联容器，其底层实现都是红黑树（RB-Tree）

map和set区别在于：

1. map中的元素是key-value（关键字—值）对：关键字起到索引的作用，值则表示与索引相关联的数据；Set与之相对就是关键字的简单集合，set中每个元素只包含一个关键字。
2. set的迭代器是const的，不允许修改元素的值；map允许修改value，但不允许修改key。

其原因是因为map和set是根据关键字排序来保证其有序性的，如果允许修改key的话，那么首先需要删除该键，然后调节平衡，再插入修改后的键值，调节平衡，如此一来，严重破坏了map和set的结构，导致iterator失效，不知道应该指向改变前的位置，还是指向改变后的位置。所以STL中将set的迭代器设置成const，不允许修改迭代器的值；而map的迭代器则不允许修改key值，允许修改value值。

3. map支持下标操作，set不支持下标操作。

map可以用key做下标，map的下标运算符[]将关键码作为下标去执行查找，如果关键码不存在，则插入一个具有该关键码和mapped_type类型默认值的元素至map中，因此下标运算符[]在map应用中需要慎用，const_map不能用，只希望确定某一个关键值是否存在而不希望插入元素时也不应该使用，mapped_type类型没有默认值也不应该使用。如果find能解决需要，尽可能用find。

9、说说vector的扩容机制，扩容后，它的内存地址会变化吗？

当 vector 的大小和容量相等（size==capacity）也就是满载时，如果再向其添加元素，那么 vector 就需要扩容。

vector 容器扩容的过程需要经历以下 3 步：

1. 完全弃用现有的内存空间，重新申请更大的内存空间；
2. 将旧内存空间中的数据，按原有顺序移动到新的内存空间中；
3. 最后将旧的内存空间释放。

因为 vector 扩容需要申请新的空间，所以扩容以后它的内存地址会发生改变。

vector 扩容是非常耗时的，为了降低再次分配内存空间时的成本，每次扩容时 vector 都会申请比用户需求量更多的内存空间（这也就是 vector 容量的由来，即 capacity>=size），以便后期使用。

10、说说C++中智能指针和指针的区别是什么？

- 智能指针

如果在程序中使用 new 从堆（自由存储区）分配内存，等到不需要时，应使用 delete 将其释放。

C++ 引入了智能指针 auto_ptr，以帮助自动完成这个过程。随后的编程体验（尤其是使用STL）表明，需要有更精致的机制。基于程序员的编程体验和 BOOST 库提供的解决方案，C++11 摒弃了 auto_ptr，并新增了三种智能指针：unique_ptr、shared_ptr 和 weak_ptr。所有新增的智能指针都能与 STL 容器和移动语义协同工作。

- 指针

C 语言规定所有变量在使用前必须先定义，指定其类型，并按此分配内存单元。指针变量不同于整型变量和其他类型的变量，它是专门用来存放地址的，所以必须将它定义为“指针类型”。

- 智能指针和普通指针的区别

智能指针和普通指针的区别在于智能指针实际上是对普通指针加了一层封装机制，区别是它负责自动释放所指的对象，这样的一层封装机制的目的是为了使得智能指针可以方便的管理一个对象的生命期。指针是一种数据类型，用于保存内存地址，而智能指针是类模板。

11、请你说说迭代器失效原因，有哪些情况

STL 中某些容器调用了某些成员方法后会导致迭代器失效。例如 vector 容器，如果调用 reserve() 来增加容器容量，之前创建好的任何迭代器（例如开始迭代器和结束迭代器）都可能会失效，这是因为，为了增加容器的容量，vector 容器的元素可能已经被复制或移到了新的内存地址。

- 序列式容器迭代器失效

对于序列式容器，例如 vector、deque，由于序列式容器是组合式容器，当当前元素的迭代器被删除后，其后的所有元素的迭代器都会失效，这是因为 vector、deque 都是连续存储的一段空间，所以当对其进行 erase 操作时，其后的每一个元素都会向前移一个位置。

- 关联式容器迭代器失效

对于关联容器，例如如 map、set，删除当前的迭代器，仅仅会使当前的迭代器失效。这是因为 map 之类的容器，使用了红黑树来实现，插入、删除一个节点不会对其他点造成影响。

12、请你说说虚函数可以是内联函数吗

1. 虚函数可以是内联函数，内联是可以修饰虚函数的，但是当虚函数表现多态性的时候不能内联。
2. 内联是在编译期建议编译器内联，而虚函数的多态性在运行期，编译器无法知道运行期调用哪个代码，因此虚函数表现为多态性时不可以内联。
3. inline virtual 唯一可以内联的时候是：编译器知道所调用的对象是哪个类，这只有在编译器具有实际对象而不是对象的指针或引用时才会发生。

13、简述一下什么是面向对象

面向对象思想是基于面向过程思想的，要说面向对象思想，先说说面向过程思想。

1. 面向过程思想

完成一个需求的步骤：首先是搞清楚要做什么，然后再分析怎么做，最后再通过代码体现。一步一步去实现，而具体的每一步都需要我们去实现和操作。这些步骤相互调用和协作，从而完成需求。在上面的每一个具体步骤中我们都是参与者，并且需要面对具体的每一个步骤和过程，这就是面向过程最直接的体现。面向过程编程，其实就是面向着具体的每一个步骤和过程，把每一个步骤和过程完成，然后由这些功能函数相互调用，完成需求。

2. 面向对象思想

面向对象的思想是尽可能模拟人类的思维方式，使得软件的开发方法与过程尽可能接近人类认识世界、解决现实问题的方法和过程，把客观世界中的实体抽象为问题域中的对象。面向对象以对象为核心，该思想认为程序由一系列对象组成。面向对象思想的特点：- 是一种更符合人类思维习惯的思想 - 可以将复杂的问题简单化 - 将我们从执行者变成了指挥者

3. 面向对象的三大特征：封装、继承、多态

封装：将事物属性和行为封装到一起，也就是 C++ 中的类，便于管理，提高代码的复用性。事物的属性和行为分别对应类中的成员变量和成员方法。

继承：继承使类与类之间产生关系，能够提高代码的复用性以及可维护性。

多态：多态意味着调用成员函数时，会根据调用方法的对象的类型来执行不同的函数。

4. 加分回答

面向过程和面向对象解决问题举例：以洗衣服为例。 1. 面向过程：接水到盆中 —— 放入衣服 —— 加入洗衣粉 —— 浸泡 —— 搓衣服 —— 过水 —— 拧干 —— 晾干 2. 面向对象：将衣服放入洗衣机 —— 加入洗衣粉 —— 开启 —— 晾干 通过例子可以发现面向对象的方式解决问题更加简单一些，但是面向对象还是基于面向过程的。

1. stm32最小系统包含哪些

STM32的最小系统的硬件组成主要有：电源电路、复位电路、时钟电路、调试接口电路、启动电路

2. ADC采集需要做什么？STM32可以直接读取吗？

ADC (模数转换器)是一种将模拟信号转换为数字信号的电路。在采集电压时，ADC 将模拟电压信号转换为数字信号，以便数字系统可以处理和分析。

ADC 采集电压的原理如下：

1. 采样: ADC首先对输入电压进行采样，即在一定时间内对电压进行测量。
2. 量化: ADC 将采样到的电压值转换为数字值。这个过程称为量化。ADC 将电压值分成若干个等级，每个等级对应一个数字值。例如，如果ADC将电压分成256个等级，则每个等级对应一个8 位数字值。
3. 编码: ADC 将量化后的数字值编码为二进制数。例如，如果ADC将电压分成256个等级，则每个数字值需要用8位二进制数表示。
4. 存储: ADC 将编码后的数字值存储在内存或寄存器中，以便数字系统可以读取和处理。总之，ADC 采集电压的原理是将模拟电压信号转换为数字信号，以便数字系统可以处理和分析。

3. 常用的串行通信总线RS485/I2C/SPI/CAN/单总线

[\(5条消息\)通信协议（485,CAN,单总线，spi，I2C）-CSDN博客](#)

连接里包含上述总线的定义、协议 和 特点，自己总结！！！！

4. CAN协议的收发，是固定主机吗，从机可以主动发数据

一般总线通信都是分主机和从机，但是CAN总线上所有的设备都是对等地位，既可以做主机也可以做从机，但是同一时刻，只允许一个主机存在。所以CAN总线可以认为是多主控制，在总线空闲的时候，所有设备都可以以主机身份发起通信。

5. 如果CAN总线同时有多个设备同时发送数据怎么办？

如果由两个以上的设备同时发起发送怎么办？CAN总线拥有仲裁机制，CAN标识符里面有优先级，总线仲裁高优先级获胜，低优先级失利，获胜方获得发送的权利

CAN总线不设定节点的地址，而是通过消息的标识符（Identifier）来区别消息，CAN标准格式标识符11位，扩展格式标识符29位。标识符格式一致，这让总线上增删节点变得更加灵活

6. 什么是通信协议？你接触过哪些通信协议？

1. 通信协议

通讯协议主要是实现两个设备之间的数据交换功能，通讯协议分**硬件层协议**和**软件层协议**。

1. 硬件层协议

决定数据如何传输问题，比如要在设备1向设备2发送0x63，0x63的二进制数为0110 0011，这8个二进制数从设备1传输到设备2，涉及到1怎么传，0怎么传的问题，这就是硬件层要解决的问题。硬件层协议目前比较多见的有RS-232、RS-485、SPI、IIC等。RS-232规定，线上的电压为x伏都表示传输的是0，y伏传输的则是1。再者，比如要选择多少条线传输数据，选择什么材质的线传输输入，这些也属于硬件层协议约束的。硬件层协议也叫接口协议，

2. 软件层协议

就指的是通讯双方在软件编码方面约定的通讯规则，比如说一个人说中文，另一个人也要说中文，两人才能进行交流；

常见的软件层协议有Modbus协议、TCP/IP协议、GPRS等；而Modbus协议可以通过串口、RS-485、以太网等来传输；

2. 常见硬件协议

1. TTL协议

通信协议MCU管脚输出TTL电平，TTL电平的意思是，当MCU管脚输出0电平时，一般情况下电压是0V，当MCU管脚输出1电平时，电压是5V。因TTL电平的是由一条信号线，一条地线产生，信号线上的干扰信号会跟随有效信号传送到接收端，使得有效信号受到干扰，

2. 485通讯实际上是把MCU出来的TTL电平通过硬件层的一个转换器芯片进行转换：

把MCU出来的一条的TTL信号经过芯片转换为两根线(线A、线B)上的信号。当MCU给转换器输入低TTL电平时，转换器会使得B的电压比A的电压高，反之，A的电压比B的电压高。

485协议规约两条电平线上差值为多少表示0或者1，电压是通过仪表可以测量得到的，所以说RS-485是硬件层协议。

485协议的接收端可能是另一个MCU，MCU管脚也只接受TTL电平，转换芯片过来的是两条线的电压，所以需要对此两条线差分电压转换为TTL电平。把TTL转为485，实质是一个集成芯片，其间无任何程序代码，纯粹硬件逻辑。同理，将485电平转为TTL也是如此。现在很多芯片把接收和转换都集成到一块IC，注意，转换器和接收器依旧是没有同时工作的，常见的转换芯片是MAX485

3. RS232

RS232协议使用不同于标准5V的信号电平传输数据，以最大程度地减少信号干扰。它以恒定速率执行异步传输，该速率与起始脉冲信号的电平同步。最长20米的距离是使用RS232接口进行可靠数据传输的极限，RS232协议使用9针串行端口，该端口可以具有公接头或母接头。该协议的最新版本称为RS232C

4. SPI

物理层上SPI 通讯使用 3 条总线及片选线，3 条总线分别为 SCK、MOSI、MISO，片选线为SS

这是一个主机的通讯时序。NSS、SCK、MOSI 信号都由主机控制产生，而 MISO 的信号由从机产生，主机通过该信号线读取从机的数据。MOSI 与 MISO 的信号只在 NSS 为低电平的时候才有效，在 SCK 的每个时钟周期 MOSI 和 MISO 传输一位数据

CPOL/CPHA 及通讯模式

SPI 一共有四种通讯模式，它们的主要区别是总线空闲时 SCK 的时钟状态（“时钟极性 CPOL”）以及数据采样时刻（“时钟相位 CPHA”）。

时钟极性 CPOL 是指 SPI 通讯设备处于空闲状态时，SCK 信号线的电平信号(即 SPI 通讯开始前、NSS 线为高电平时 SCK 的状态)。CPOL=0 时，SCK 在空闲状态时为低电平，CPOL=1 时，则相反。

时钟相位 CPHA 是指数据的采样的时刻，当 CPHA=0 时，MOSI 或 MISO 数据线上的信号将会在 SCK 时钟线的“奇数边沿”被采样。当 CPHA=1 时，数据线在 SCK 的“偶数边沿”采样

5. I2C

一个 I2C 总线只使用两条总线线路，一条双向串行数据线(SDA)，一条串行时钟线(SCL)

在一个 I2C 通讯总线中，可连接多个 I2C 通讯设备，支持多个通讯主机及多个通讯从机

每个连接到总线的设备都有一个独立的地址，主机可以利用这个地址进行不同设备之间的访问，

I2C 设备空闲时，会输出高阻态，所有设备都空闲时，有总线上的上拉电阻拉高双线

标准模式传输速率为 100kbit/s，快速模式为 400kbit/s，高速模式下可达 3.4Mbit/s

6. CAN

CAN节点通过CAN_High、CAN_Low两根线接入CAN总线网络。CAN收发器将CAN控制器的CAN_TX输出的逻辑值0、1转换为差分信号，通过CAN_High、CAN_Low向总线输出

CAN总线为“线与”逻辑，0为显性，1为隐性，只要有一个节点输出0，总线即表现为0，所有节点都输出1时，总线才表现为1

CAN总线以 帧为单位，发送消息，并且以ID来实现消息的优先级并完成总线仲裁

总线仲裁

当多个节点同时发送数据报文时，节点通过仲裁段来参与总线的竞争。节点向总线发送数据的同时会检测总线的电平状态，如果采集电平和发送电平一致则继续发送，否则转为接收者

7. 电流和电压的关系

电流和电压的关系是什么 成正比关系。按照欧姆定律: $I=U \div R$,可以看出,若没有电阻,电压与电流就失去意义了。而且如果电路中未使用到电器,无法形成回路,即使有电压,电流也不存在。因此得出电流是由电压产生的,有电流就必须要有电压。

8. 单片机选的哪一款，工作频率是多少

STM32F407ZET6，最高频率168MHz，Flash大小为512M，基于Cortex-M4

9. MCU如何选型

1、统计有哪些外设

如果一个系统里面有多MCU，这个时候要先分配好，如温度挂1# MCU，蓝牙挂2# MCU，等等。

2、确定外设的通信接口

有的外设支持多种接口，如I2C和SPI，这个时候要根据外设的数据量、开发工作量、以往产品经验、产品定义、功耗等多个方面来确定好接口。

3、确定外设通信接口的电平

某些外设通信接口的电平，参考的不是电源电压。如供电2.8V，通信接口是1.8V，给芯片VDDIO的电压，I2C根据外部的上拉源电压等等，这些先确定好。

4、确定外设的供电范围

某些外设的电流很小，供电电压和MCU的电压一致，我们可以直接IO驱动，如G-sensor。

某些外设的电压需要用电源芯片转换，这个时候就要考虑增加GPIO（电源控制脚）。

5、确定外设的GPIO数量

要根据规格书上的电路，先大致确定，如外设的电源需不需要控制，中断脚，唤醒脚，需不需要电平转换，电平转换需不需要加GPIO使能控制。

6、确定外设GPIO的种类

哪些用一般GPIO就可以，哪些需要用带中断的GPIO，哪些可以配置成PWM等等。

7、确定ADC的数量

如电池电压检测，ID识别等。

8、I2C接口外设确定多从机

I2C支持一主多从，确定好哪些外设挂在同一个I2C下面，看地址是否冲突。

9、统计外设占用的内存和flash大小

提前需求厂家的帮助，各外设大致所占用的内存和flash的大小。这一点非常重要，以免到后期出现尴尬局面。

10. CAN通信描述一下

CAN总线是一种分布式的控制总线，它的网络由很多CAN节点构成，每个节点均由一个MCU（微控制器）、一个CAN控制器和一个CAN收发器构成，然后使用双绞线连接到CAN网络中。另外，在CAN总线的起止端都有一个120Ω的终端电阻，用来做阻抗匹配，以减少回波反射

帧的种类

通信是通过以下 5 种类型的帧进行的：

帧类型	帧用途
数据帧	用于发送单元向接收单元传送数据的帧
遥控帧	用于接收单元向具有相同 ID 的发送单元请求数据的帧
错误帧	用于当检测出错误时向其它单元通知错误的帧
过载帧	用于接收单元通知其尚未做好接收准备的帧
间隔帧	用于将数据帧及遥控帧与前面的帧分离开来的帧

其中，数据帧和遥控帧有标准格式和扩展格式两种格式。

标准格式有11 个位的标识符（ID），扩展格式有29 个位的ID。

- (1) 帧起始。表示数据帧开始的段。
- (2) 仲裁段。表示该帧优先级的段。
- (3) 控制段。表示数据的字节数及保留位的段。
- (4) 数据段。数据的内容，一帧可发送0~8个字节的数据。
- (5) CRC段。检查帧的传输错误的段。
- (6) ACK段。表示确认正常接收的段。
- (7) 帧结束。表示数据帧结束的段。数据帧和遥控帧的不同

- 遥控帧的 RTR 位为隐性位，没有数据段。
- 没有数据段的数据帧和遥控帧可通过 RTR 位区别开来。
- 遥控帧没有数据段，数据长度码该如何表示？
- 遥控帧的数据长度码以所请求数据帧的数据长度码表示。
- 没有数据段的数据帧有何用途？
- 例如，可用于各单元的定期连接确认/应答、或仲裁段本身带有实质性信息的情况下。

优先级的决定

在总线空闲态，最先开始发送消息的单元获得发送权。

多个单元同时开始发送时，各发送单元从仲裁段的第一位开始进行仲裁。连续输出显性电平最多的单元可继续发送。